



汇编程序设计课程实验报告

题目 分支和循环程序设计实验(2)

姓 名：王子琪

(学 号)：22920182204309

院 系：信息学院

专 业：人工智能

年 级：2018 级

2019 年 4 月 12 日

一、实验目的

1. 熟悉掌握汇编语言程序设计方法，能灵活使用各种跳转指令，
进一步掌握循环、分支程序设计的方法
2. 熟悉 DOS 功能调用，包括字符的输入/输出，以及字符串的输出
3. 熟悉使用 emul8086 的 Debug 调试功能

二、实验思路和解答

1. 在 AX 寄存器中给定一个无符号数，屏幕中输出其十进制的数值

例如：

AX : 1234H

输出：4660

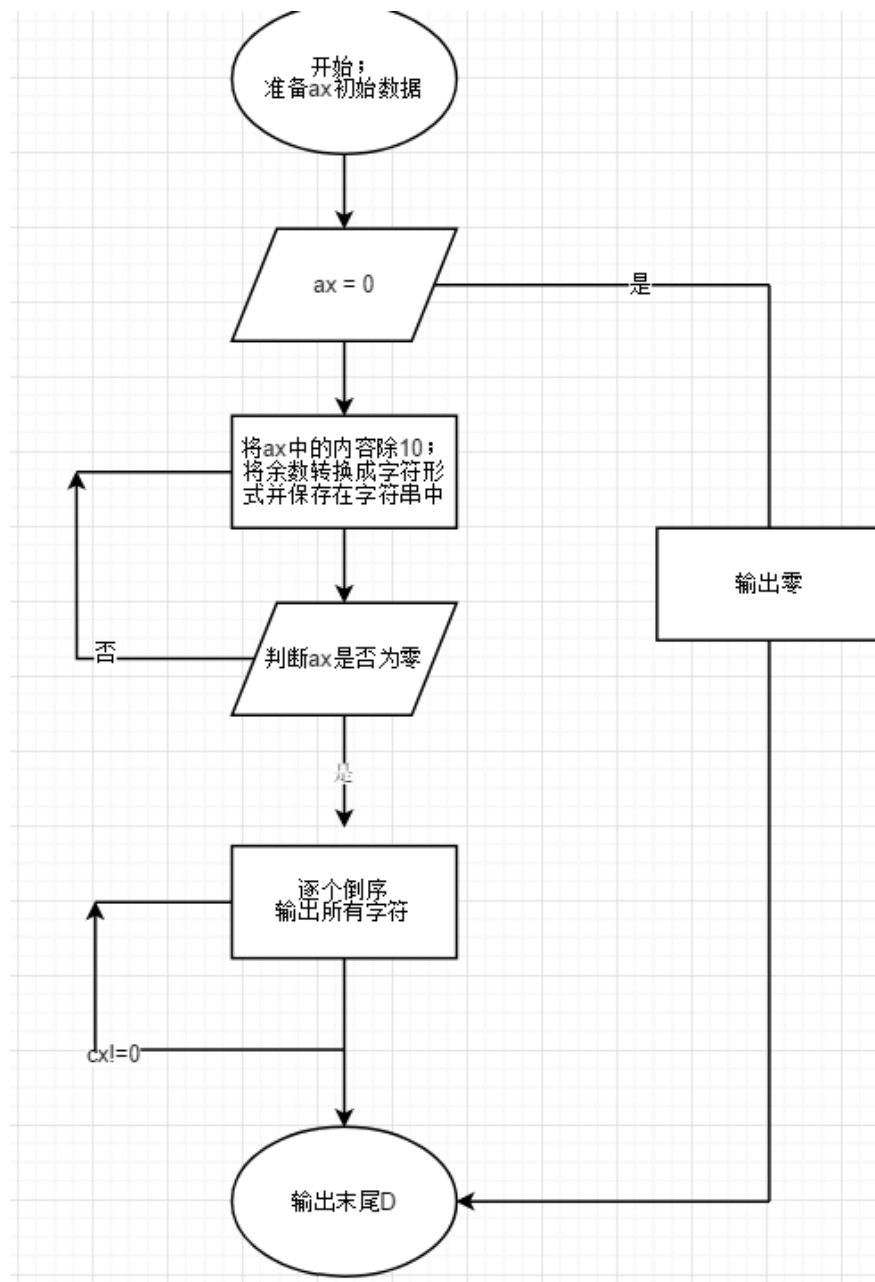
要求输出的数据首位非 0，即不能输出 004660 这种形式

挑战：输出 16 进制数？

思路：

利用除法得到每一位，存储起来，最后逆序输出

流程图：



代码:

```
; multi-segment executable file template.

data segment
    ; add your data here!
    k dw 10
    mess1 db 0      ; 用来储存十进制数的倒序
ends

stack segment
    dw 128 dup(0)
ends

code segment
start:
    ; set segment registers:
    mov ax, data
    mov ds, ax
    mov es, ax

    mov ax, 0FFFFH ; 预先设置ax中的内容
    cmp ax, 0
    jz zero        ; 如果ax是零的话直接预先处理
    mov cx, 0      ; cx 用来记录转换后的十进制的位数，辅助输出
    lea di, mess1
; 转换十六进制为十进制，并以字符形式存储在mess1中
loop1:
    cmp ax, 0
    jz prin       ; 脱离循环的为一条条件
    div k
    add dl, 48    ; dl中存储的是余数，将数字形式转换为字符形式存储到首地址为mess1的字符串中
    mov es:[di], dl
    mov dl, 0
    inc di        ; 字符串指针移向下一个空位
    inc cx        ; 代表十进制位数多1

    lea di, mess1
; 转换十六进制为十进制，并以字符形式存储在mess1中
loop1:
    cmp ax, 0
    jz prin       ; 脱离循环的为一条条件
    div k
    add dl, 48    ; dl中存储的是余数，将数字形式转换为字符形式存储到首地址为mess1的字符串中
    mov es:[di], dl
    mov dl, 0
    inc di        ; 字符串指针移向下一个空位
    inc cx        ; 代表十进制位数多1
    jmp loop1
; 倒序逐个输出mess1中的内容
prin:
    dec di        ; 到达prin模块，必定是经历了loop1至少1轮，即ax!=0, 所以此时di实际指向在有意义的字符的下一位，所以要减一
loop2:
    cmp cx, 0
    jz exit       ; cx是之前创建的计数器
    mov dl, es:[di] ; 逐位输出
    mov ah, 02h
    int 21h
    dec cx
    dec di
    jmp loop2

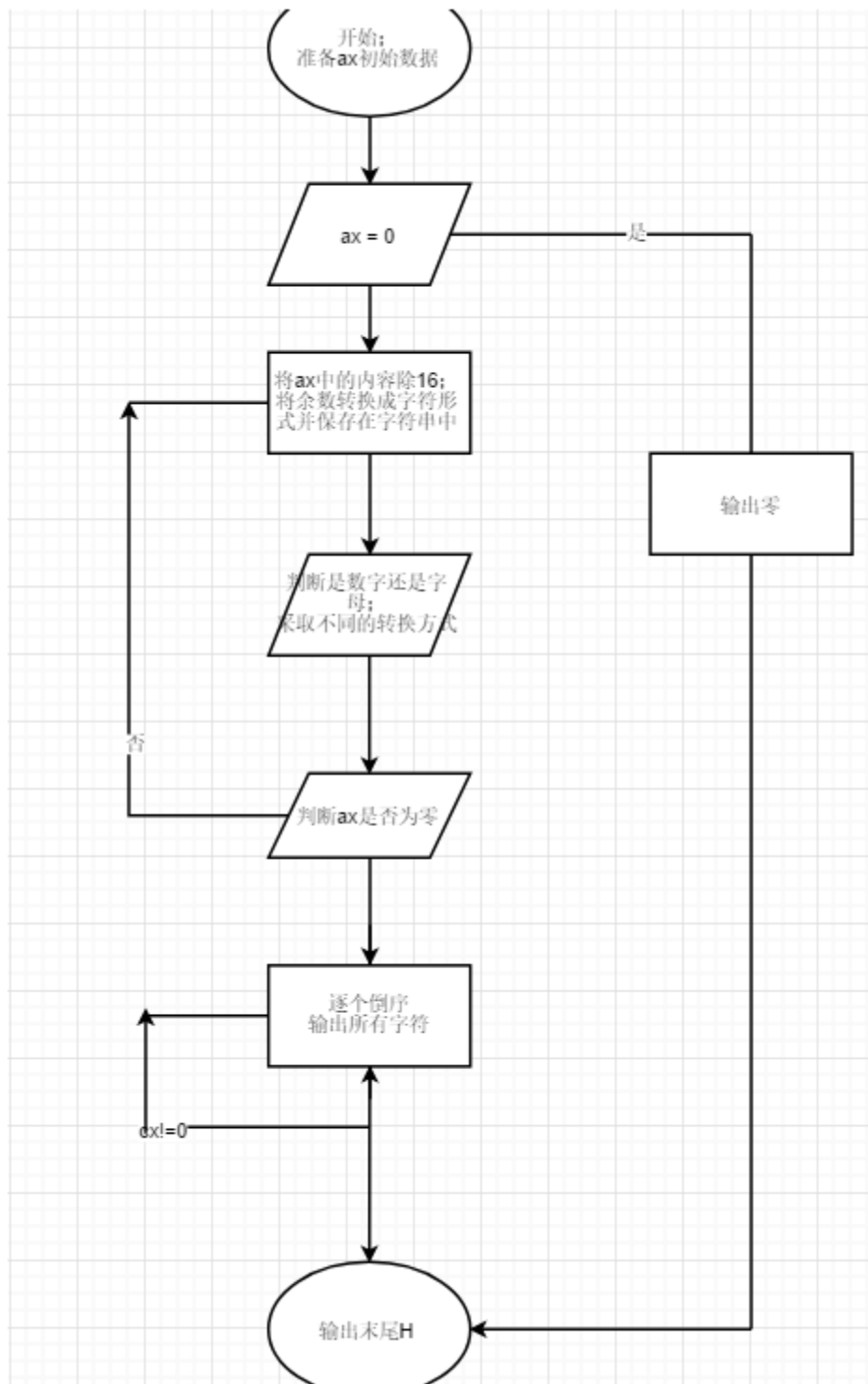
; ax=0 的情况单独处理
zero:
    mov dl, '0'
    mov ah, 02h
    int 21h
    jmp exit
exit:
```

结果：ax 中内容为 0FFFFH



挑战：输出 16 进制数？

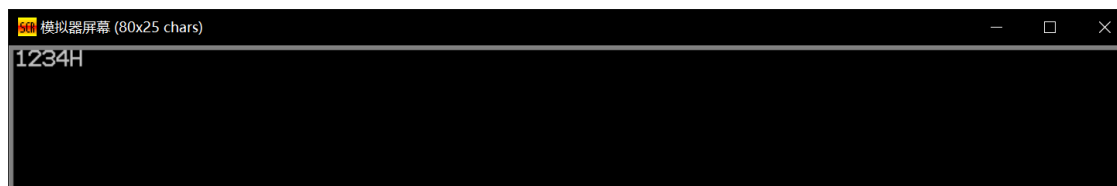
流程图：



代码:

```
01 ; multi-segment executable file template.
02
03 data segment
04     ; add your data here!
05     k dw 16
06     mess1 db 0 ;用来储存十进制数的倒序
07 ends
08
09 stack segment
10     dw 128 dup(0)
11 ends
12
13 code segment
14 start:
15 ; set segment registers:
16     mov ax, data
17     mov ds, ax
18     mov es, ax
19
20     mov ax, 1234H ;预先设置ax中的内容
21     cmp ax, 0
22     jz zero ;如果ax是零的话直接预先处理
23     mov cx, 0 ;cx 用来记录转换后的十进制的位数, 辅助输出
24     lea di, mess1
25 ;转换十六进制为十进制, 并以字符形式存储在mess1中
26 loop1:
27     cmp ax, 0
28     jz prin ;脱离循环的为一条条件
29     div k
30     cmp dl, 10 ;判断是字母还是数字的输出, 采取不同的字符转换方是
31     js num
32     jmp char ;
33 num:
34     add dl, 48 ;dl中存储的是余数, 将数字形式转换为字符形式存储到首地址为mess1的字符串中
35     mov es:[di], dl
36     jmp later
37 char:
38     add dl, 55
39     mov es:[di], dl
40 later:
41     mov dl, 0
42     inc di ;字符串指针移向下一个空位
43     inc cx ;代表十进制位数多1
44     jmp loop1
45 ;倒序逐个输出mess1中的内容
46 prin:
47     dec di ;到达prin模块, 必定是经历了loop1至少1轮, 即ax!=0, 所以此时di实际指向在有意义的字符的下一位, 所以要减一
48 loop2:
49     cmp cx, 0 ;cx是之前创建的计数器
50     jz exit
51     mov dl, es:[di] ;逐位输出
52     mov ah, 02h
53     int 21h
54     dec cx ;
55     dec di ;
56     jmp loop2
57
58 ;ax=0 的情况单独处理
59 zero:
60     mov dl, '0'
61     mov ah, 02h
62     int 21h
63     jmp exit
64 exit:
65     mov dl, 'H'
66     mov ah, 02h
67     int 21h
68 ends
```

结果:



实验二：编写一段程序，要求根据用户键入的月份数在终端上显示该月的英文缩写名

例如

input : 8

output: OCT

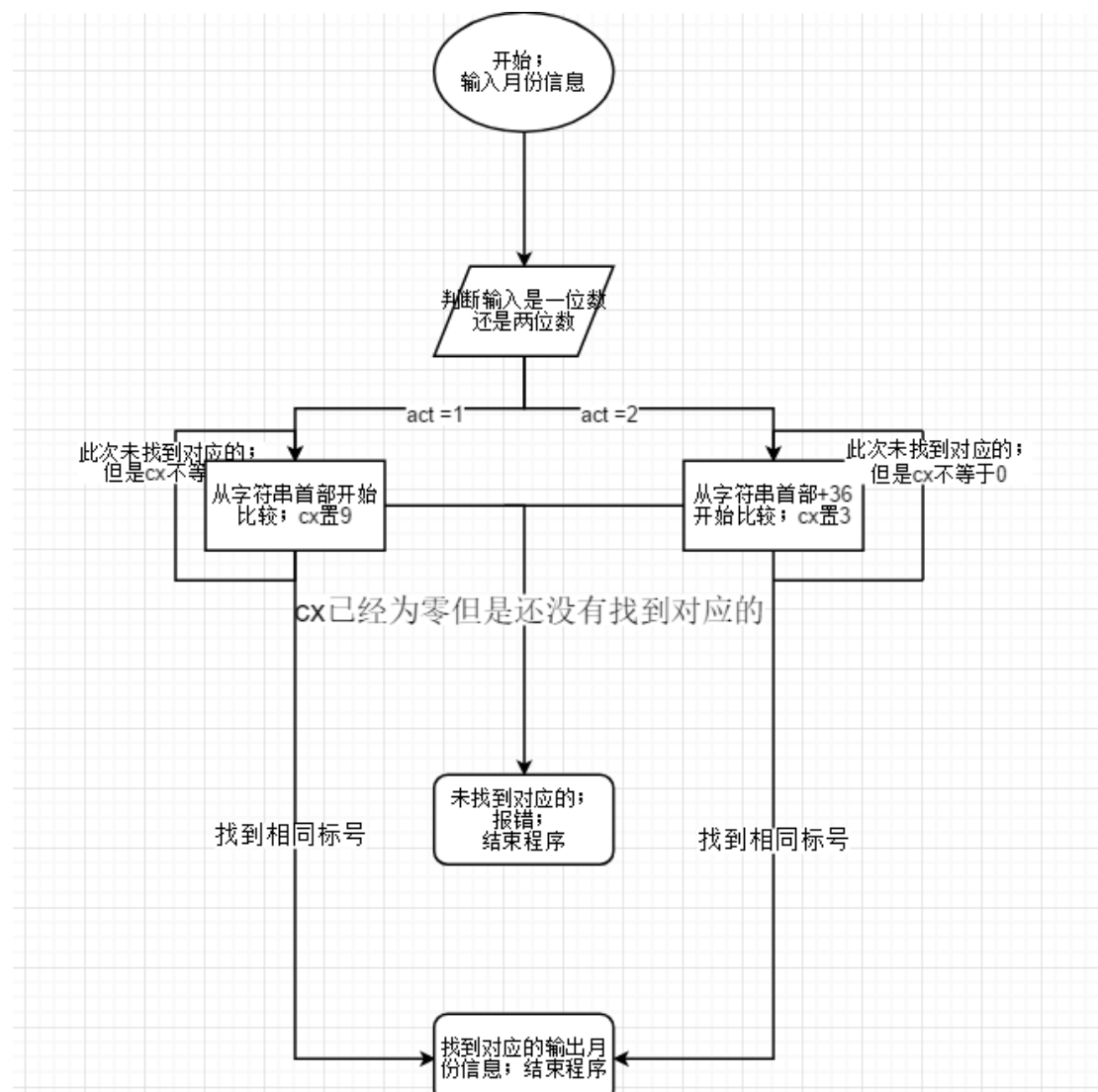
input : 12

output: DEC

思路：

通过 10 号中断获得输入，将输入分为一位两位两种情况，分别与制作的月份表进行对比，分别采取字节对比和字对比，当发现相同的则输出相应英文，若无则报错

流程图：



代码：

```
01 ; multi-segment executable file template.
02
03 data segment
04     ; add your data here!
05     mess1 db "wrong input", 13, 10, '$'
06     num label byte
07         max db 3
08         act db 0
09         stoken db 3 dup(0)
10     stoktab db '1', 'JAN'           ;月份表
11             db '2', 'FEB'
12             db '3', 'MAR'
13             db '4', 'APR'
14             db '5', 'MAY'
15             db '6', 'JUN'
16             db '7', 'JUL'
17             db '8', 'AGU'
18             db '9', 'SEP'
19             db '10', 'OCT'
20             db '11', 'NOV'
21             db '12', 'DEC'
22     mess2 db 3 dup(20h), 13, 10, '$'
23 ends
24 code segment
25 start:
26 ; set segment registers:
27     mov ax, data
28     mov ds, ax
29     mov es, ax
30
31     lea dx, num       ;存储输入
32     mov ah, 0ah
33     int 21h
34     mov al, stoken    ;将输入结果放在ax中
```

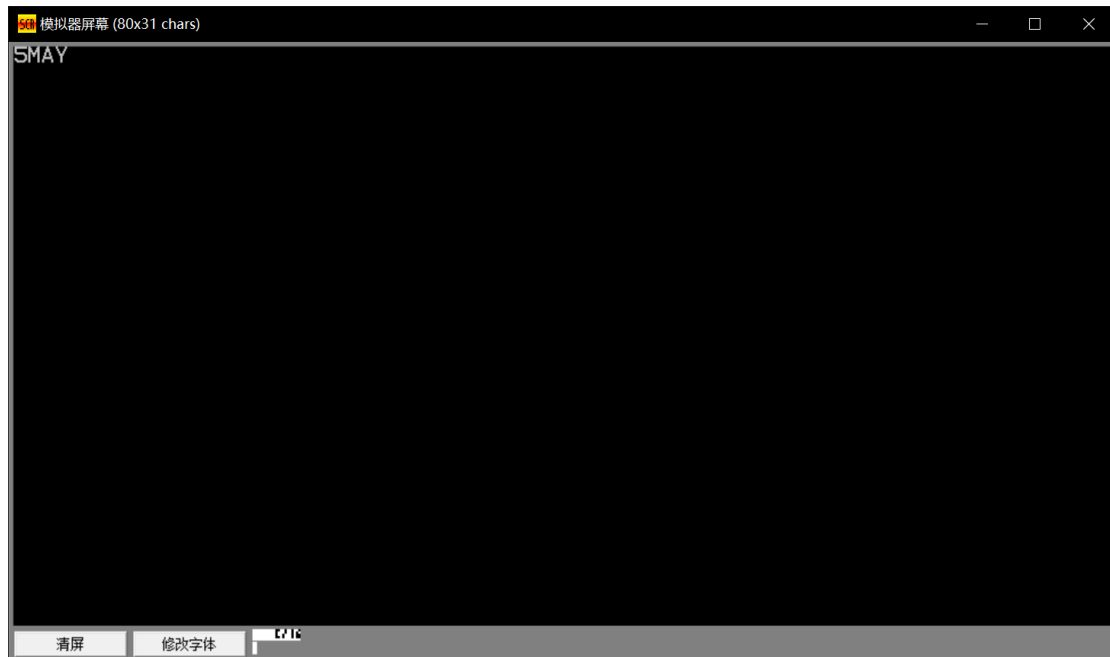
```

35     mov ah, (stokn+1)
36
37     mov cx, 9
38     lea si, stoktab ;因为有可能输入一位或者两位，我们选择分类讨论，先假定输入为1为进行预处理
39
40     mov dl, act      ;验证输入为一位还是两位
41     cmp dl, 2
42
43     js loop1         ;如果是一位的话，则进入1号循环比较
44     mov cx, 3
45     add si, 36        ;如果是两位的话，需要的预处理
46 ;loop2 和 later2都是输入为两位的处理
47 loop2:
48     cmp cx, 0
49     jz wrong         ;进入wrong代表输入错误，例如1-12的其他东西
50     cmp ax, word ptr [si]
51     jnz later2       ;并不相同，则需要进行下一次循环的准备
52     add si, 2        ;相同的话，将si指针指向字符串
53     jmp prin
54
55 later2:
56     dec cx
57     add si, 5
58     jmp loop2
59 ;loop1 和 later1 都是输入为一位的处理
60 loop1:
61     cmp cx, 0
62     jz wrong
63     cmp al, byte ptr [si]
64     jnz later1
65     add si, 1
66     jmp prin
67 later1:
68     dec cx
69     add si, 4
70     jmp loop1
71 ;找到了对应的数字，则输出相应的月份
72 prin:
73     mov cx, 3
74     lea di, mess2
75     rep movsb
76
77     lea dx, mess2
78     mov ah, 09h
79     int 21h
80
81     jmp exit
82 ;未找到对应的数字，则报错
83 wrong:
84     lea dx, mess1
85     mov ah, 9h
86     int 21h
87     jmp exit
88
89 exit:
90
91 ends
92
93 end start ; set entry point and stop the assembler.

```

结果:

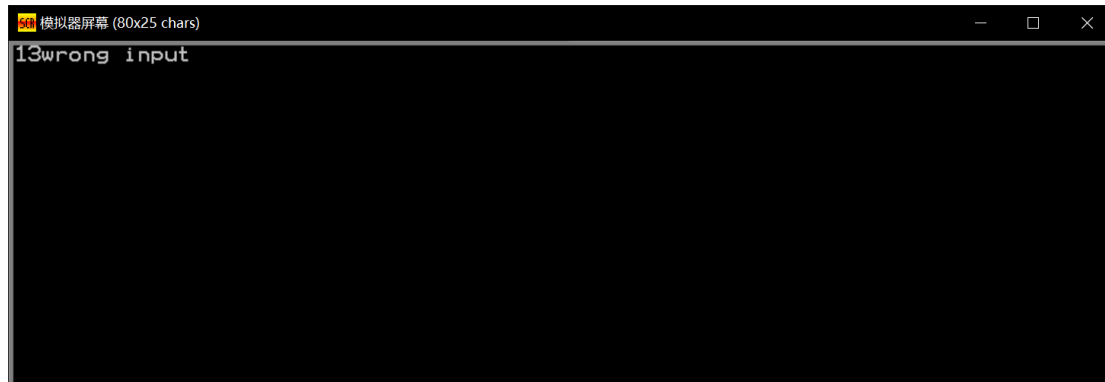
输入 5 (5 为输入内容, 不是显示内容)



输入 11



输入 13

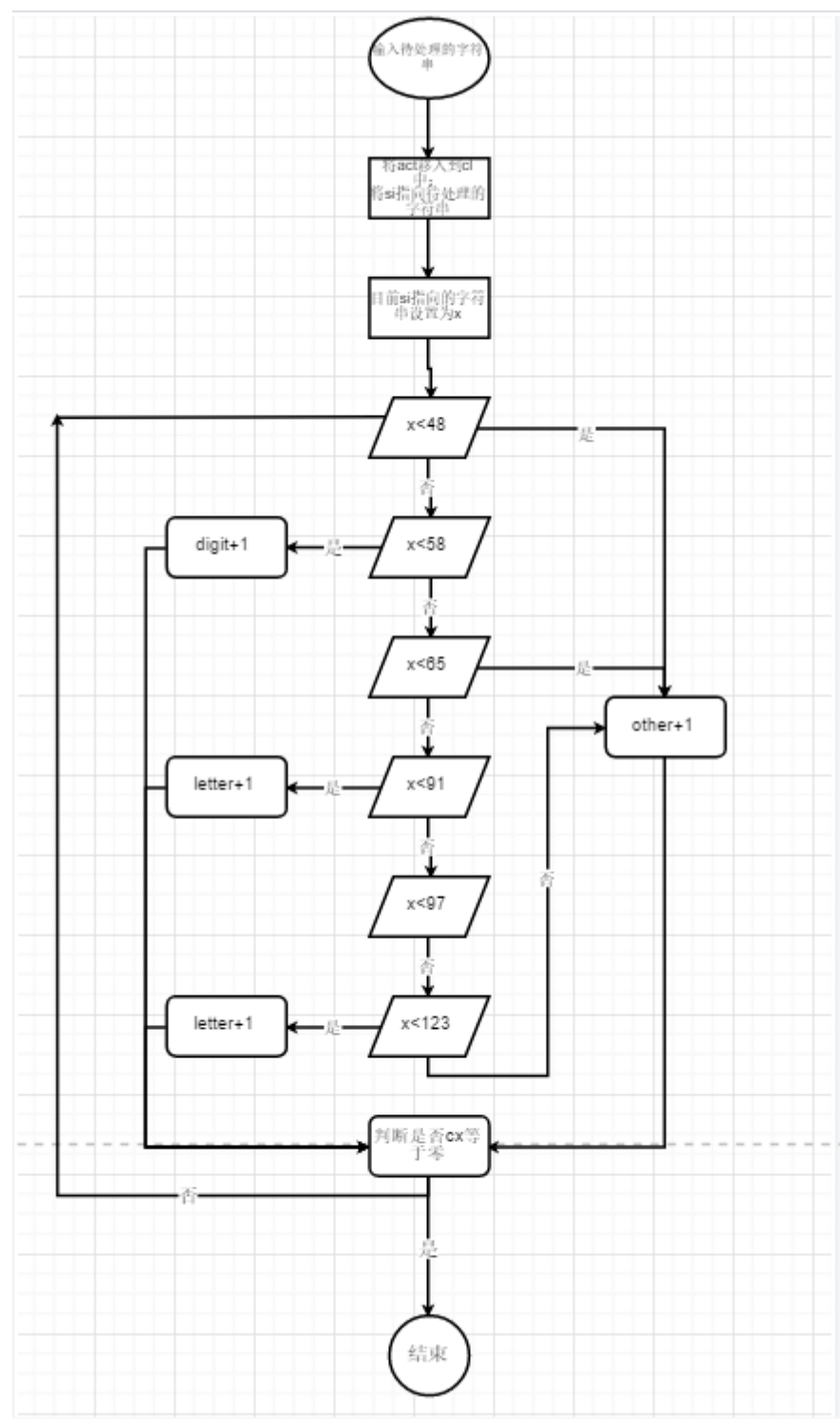


实验三：接受用户键入的一行字符串（个数不超过 80 个, 用回车键结束），要求统计这一字符串中字母、数字、其他字符出现的次数，将结果存入以 letter、digit 和 other 为名的存储单元中

思路：

使用 label 存储，逐个处理即可。

流程图：



代码:

```
01 ; multi-segment executable file template.
02
03 data segment
04     num label byte
05         max db 81
06         act db 0
07         stokn db 81 dup(0)
08     letter db 0
09     digit db 0
10     other db 0
11 ends
12
13
14
15 code segment
16 start:
17 ; set segment registers:
18     mov ax, data
19     mov ds, ax
20     mov es, ax
21
22     lea dx, num
23     mov ah, 0ah
24     int 21h
25
26     mov cl, act
27     lea si, stokn        ;预处理
28
29 loop1:
30     cmp cx, 0            ;cx = 0意味着处理完了所有的输入
31     jz exit
32     cmp [si], 48         ;根据ascii值来进行分流
33     js other1
34     cmp [si], 58
```

```

35     js digit1
36     cmp [si], 65
37     js other1
38     cmp [si], 91
39     js letter1
40     cmp [si], 97
41     js other1
42     cmp [si], 123
43     js letter1
44     jmp other1
45 ;检测到是数字
46 digit1:
47     inc digit
48     jmp later
49 ;检测到是字母
50 letter1:
51     inc letter
52     jmp later
53 ;检测到是其他
54 other1:
55     inc other
56     jmp later
57 later:
58     inc si
59     dec cx
60     jmp loop1
61
62 exit:
63 ends
64
65 end start ; set entry point and stop the assembler.

```

结果:

输入:



输出:

LETTER	04h
DIGIT	02h
OTHER	06h

四、实验心得

1. 更加熟悉循环和分支
2. 对于输入长字符串且需要知道位数，可以选择 `label`
3. 熟悉使用指针 `si` 来处理长字符串
4. 更加熟悉基本指令